COURSEWORK SPECIFICATION

ECM2414 – Software Development

Module Leader: Prof. Solomon Oyelere

Academic Year: 2026/27

Title: **Software Development Continuous Assessment**

Handed out date: **23rd September 2026**

Submission deadline: **20th October 2026**

This assessment contributes **40%** of the total module mark and assesses the following **intended learning outcomes**:

- Use a software design and development method which incorporates both formal and informal techniques appropriately.
- Design and implement rigorous testing frameworks for software and be aware of the inherent limitations of tests developed.
- Deploy advanced object-oriented language concepts and techniques.
- Develop multi-program software systems.
- Follow the phases of software development.
- Recognise and evaluate different development practices, and judge their appropriateness for a specific development problem.
- Analyse and break down a problem into constituent parts.
  Compare and critically contrast different potential solutions to a problem.
- Follow the pair-programming development approach used widely in industry.

Plagiarism

This is a pair assessment, and you are reminded of the University's regulations on collaboration and plagiarism. You must avoid plagiarism, collusion, and any academic misconduct behaviours. Further details about academic honesty and plagiarism can be found at https://ele.exeter.ac.uk/course/view.php?id=1957.

Generative AI

This assessment is AI-prohibited – where the use of GenAI tools is prohibited as their use prevents achievement of the module learning outcomes. This assessment is designed to establish independent competence in object-oriented design, testing and concurrency before these activities are delegated to AI tools. Rather than relying on AI-detection scores, the assessment requires development-process evidence and an individual code defence as direct evidence of your development process and understanding. You can find further information in the university's policies around using AI in assessed work

Instructions

In this assignment you will implement an application using the various techniques that you are learning about during this module. The core artefact remains a pair-programming task, reinforcing programming as a team exercise and the software-development process. After the pair submission, each student will also complete a short individual code defence/live exercise, as described in Section 2. Please find the pair information on ELE (in the Assessment tile). Any changes to the members of a pair need to be approved by the module leader, otherwise a mark of zero for the relevant teams may be applied.

1. Task Specification

You will develop, in Java, a *multi-threaded* card playing simulation. Within your design you will need to implement (at least) a thread-safe Card class and a thread-safe Player class (depending upon your design, you may also implement additional classes, for instance a CardDeck class). You will also develop an executable CardGame class.

The game has *n* players, each numbered 1 to *n* (which for clarity in the illustration below are named player1, player2, …, player*n*), with *n* being a positive integer, and *n* decks of cards, again, each numbered 1 to *n* (which for clarity in the illustration below are named deck1, deck2, … , deck*n*). Each player will hold a hand of 4 cards. Both these hands and the decks will be drawn from a **pack** which contains 8*n* cards. Each card has a face value (denomination) of a non-negative integer[1].

The decks and players will form a ring topology (see illustration in the figure below for the case where *n* = 4). At the start of the game, each player will be distributed four cards in a round-robin fashion, from the top of the pack, starting by giving one card to player1, then one card to player2, etc. After the hands have been distributed, the decks will then be filled from the remaining cards in the pack, again in a round-robin fashion.

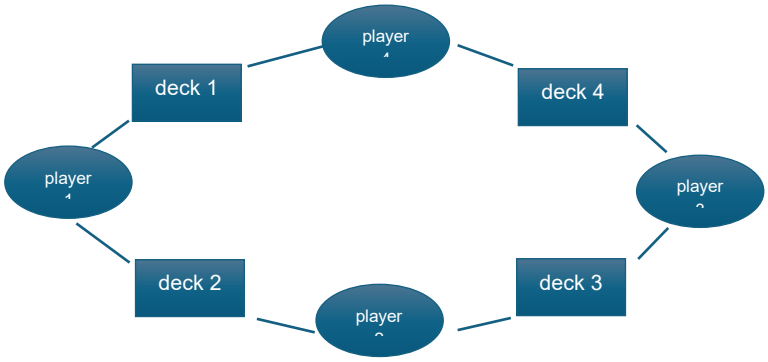


Figure 1: Topological relationship of the game players and card decks, in the situation where *n* = 4.

Deck definition. A **deck** is an ordered, thread-safe container (FIFO) holding card denominations from the shared 8n pack. Players **draw from the top (front)** and **discard to the bottom (back)** of decks as specified by the ring topology (player *i* draws from deck *i* and discards to deck *i+1*). Draw and discard together are treated as **one atomic action**. At

[1] It is legal for the face value of a card exceeding *n*.

game end, **each player holds exactly four cards**, and each deck's **final contents** are written to deckX_output.txt (deck sizes may vary).

To win the game, a player needs four cards of the same value in their hand. If a player is given four cards which are all the same value at the start of the game, they should immediately declare this (by their thread printing "Player *i* wins", where *i* should be replaced with the player index), that player thread should then notify the other threads, and exit[2].

If the game is not won immediately, then the game progresses as follows: each player picks a card from the top of the deck to their left and discards one to the bottom of the deck to their right[3]. This process continues until the first player declares that they have four cards of the same value, at which point the game ends[4].

1.1 Game playing strategy

If a player does not start with a winning hand, they will implement a simple game strategy, as specified below (note, the strategy is *not* optimal).

Each player will prefer certain card denominations, which reflect their index value, e.g., player1 will prefer 1s, player2 will prefer 2s, etc. After drawing a card from their left, a player will discard one of their cards to the deck on their right, (e.g. player1 will draw from deck1 and discard to deck2). The card they discard must display a value which is **not** of their preferred denomination. Additionally, a player **must not** hold onto a non-preferred denomination card indefinitely, so you must implement your Player class to reflect this restriction (otherwise the game may stagnate).

1.2 Solution development

You will need to implement an executable class called CardGame, whose main method requests via the command line (terminal window) the number of players in the game (i.e. '*n*'), and on receiving this, the location of a valid input pack. The screenshot below shows how this works (note, I put the pack file in the same directory of CardGame, so there is no path needed in the example shown below.)

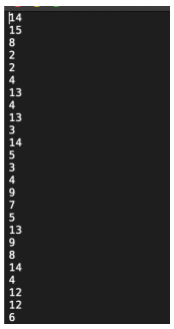
```
Yuleis-Air:example_answer_CA1 yw433$ java CardGame
Please enter the number of players:
4
Please enter location of pack to load:
four.txt
```

A valid input **pack** is a plain text file, where each row contains a single non-negative integer value, and has 8*n* rows. The screenshot below shows a partial pack

[2] There is a chance that, two or more players are given four cards with the same value at the start of the game. You don't need to handle this situation in your development.

[3] By multi-threading, players should NOT play the game sequentially, i.e., NOT in a way that, when one player finishes actions another player starts.

[4] By multi-threading, there is a chance that, two or more players have their four cards of the same value at the same time. You don't need to handle this situation in your development.



After reading the input pack, the CardGame class should distribute the hands to the players, fill the decks and start the required threads for the players. If the pack is **invalid**, the program should inform the user of this, and request a valid pack file[5].

If a player does not start with a winning hand, as a player processes their hand, each of its actions should be printed to an output file which is named after that particular player (i.e. the output file for the first player should be named player1_output.txt)[6]. In game actions should be printed to the file in a similar form to the following example:

player 1 draws a 4 from deck 1

player 1 discards a 3 to deck 2

player 1 current hand is 1 1 2 4

Additionally, at the start of the game the first line of the file should give the hand dealt, e.g.

player 1 initial hand 1 1 2 3

and at the end of the game the last lines of the file should read either:

player 1 wins

player 1 exits

player 1 final hand: 1 1 1 1

if for instance player 1 wins, or

player 3 has informed player 1 that player 3 has won

player 1 exits

player 1 hand: 2 2 3 5

in the case where player 3 has won (and the values displayed match the hands and decks concerned).

There should also be a message printed to the terminal window (as is the case when a player wins immediately), i.e. if the 4th player wins, then

player 4 wins

[5] The game should not start until there are valid inputs for the 'number of players' and the 'pack' file.

[6] Each player should have its own output file, NOT a combined output file for all players.

should be printed to the screen.

There should only be one player declaring it has won for any single game[7]. If the game is won immediately (a winning hand is initially dealt), the output files should still be written for all players (although each file will only contain four lines). In addition to the player output files, there should also be *n* deck output file written at the end of the game (named, e.g. deck1_output.txt), which should contain a single line of text detailing the contents of the deck at the end of the game, e.g.

deck2 contents: 1 3 3 7

The combination of a card draw, and a discard should be treated as a single atomic action. Therefore, at the end of the game every player should hold four cards. The program developed should follow the object-oriented paradigm.

You must use a version-control system (for example, Git) during development. Your submitted process evidence should show a coherent development history, including selected meaningful commits and the evolution of implementation and testing. Marks are awarded for the quality and coherence of this evidence, not for the number of commits.

2. Submission Requirements

The pair submission must include the code part (cards.jar and cardsTest.zip), the design rationale/report, and the required development-process evidence in a single ZIP upload. Marks cannot be awarded for a required component that is omitted. After the pair submission, each student must complete the individual code defence/live exercise, which is worth 20% of the coursework mark and is assessed individually. You need to ZIP the pair-submission components into one file and submit it electronically using ELE by 12 noon on the hand-in date.

The Code Part. Your code part includes the following two files (i.e., cards.jar and cardsTest.zip).

- A copy of your finished classes in an executable *jar* file named cards.jar. The jar file should include **both** the bytecode (*.class*) and source files (*.java*) of your submission.
- A copy of your finished classes, associated test classes and test suites, and any supporting files, in a zip file named cardsTest.zip. The zip file should include both the bytecode (.class) and source files (.java) of your submission (plus any testing files the tests rely upon), and a README file giving exact instructions for compiling and running the test suite, including the JDK/JUnit version and any required dependencies.

The Written Evidence Part. The written submission, with minimum 2 cm margins and 11-point text, should contain the items listed below.

[7] By multi-threading, there is a chance that, two or more players declare they have won the game simultaneously. You don't need to handle this situation in your development. You can simply give it another run.

- A design rationale and critical report of no more than three sides of A4. It should explain the architecture of the production code, key object-oriented and concurrency decisions, alternatives considered, the testing strategy, known limitations or performance issues, and how the implementation satisfies the specification. The emphasis should be on rationale and evaluation rather than a description of individual methods.
- Within the report, explain the design of your tests and the limitations of your testing approach, and state whether you are using JUnit 4.x or 5.x. High marks require tests of behaviour and invariants, not primarily tests of getters/setters or superficial code coverage. Relevant examples include FIFO deck behaviour, maintaining exactly four cards in each player hand after a completed atomic action, conservation of cards, preferred-denomination discard behaviour, valid and invalid input handling, immediate-win detection, clean termination after a winner, and generation of the required output files. Code coverage alone is not sufficient evidence of a rigorous test suite.
- A development-process and pair-programming record of no more than one side of A4. It should include the date, time and duration of pair-programming sessions, the driver/navigator roles taken, and 3-5 significant version-control commits (commit identifier/date plus a brief description of what changed, was tested, or was debugged). The evidence should show an authentic, iterative development process. Marks are based on the quality and coherence of the evidence, not on the number of commits. Each pair-programming entry should be agreed by both members.

Individual Code Defence / Live Exercise (20%). After the pair submission, each student will take part in a short 5-8 minute individual conversation with a member of the teaching team. You may be asked to explain part of the submitted architecture or code, reason about thread safety or a game scenario, diagnose a small defect, or describe or make a small change or test. The purpose is to verify your understanding of the work submitted. This component is assessed individually, so the two members of a pair may receive different overall coursework marks. You may refer to your submitted code during the conversation.

3. Some Questions You May Have

1. Which card should a player choose to discard?

Answer: You can randomly choose one which is not the preferred denomination to discard.

2. Can a player who is given four cards which are all the same value at the start of the game, but the value is not their preferred denomination, still win?

Answer: Yes, it still can win the game.

3. How many output files from the production code?

Answer: 2n, where n is the number of players.

4. Do I need to use Javadoc?

Answer: No, you don’t need to.

5. My code reads in a valid pack file which doesn’t contain the cards that allow a winning hand. What should I do?

Answer: It’s normal. You don’t need to do anything. The code and pack are two separate things. You need to make sure your code is correct, which means if it takes in a valid pack that allows a winning hand, it can output a winning hand.

6. Do I need to test private methods?

Answer: No. Tests should normally exercise observable behaviour through the public interface. You are not required to test private methods directly. If important behaviour is difficult to test, reconsider the design or test it through the public methods that depend on it. High marks come from well-chosen behavioural, invariant, boundary and failure tests, not from directly invoking every method.

Your CA will be marked based on the criteria in Section 4.

4. Marking criteria

This assessment will be marked using the following criteria.

Marking Scheme	Description	Mark
Pair-submitted components (80%)		
Implementation correctness, OO design and concurrency	The program satisfies the required functional behaviour, uses a coherent object-oriented design with clear management of state, is thread-safe, correctly implements the atomic draw/process/discard action, and produces and accepts input/output in the specified formats.	30%
Testing, robustness and software quality	JUnit tests are well formulated and test behaviour, invariants, boundary/error cases and concurrency-relevant behaviour. Exception handling is robust and the code is maintainable. High marks require meaningful behavioural and invariant testing; code coverage alone is not sufficient.	20%
Design rationale and critical report	The report is well structured and critically explains the architecture, object-oriented and concurrency decisions, alternatives considered, testing strategy, limitations and alignment with the specification. It prioritises rationale and evaluation over method-by-method description.	15%
Development process and pair-programming evidence	The submitted evidence demonstrates an authentic, iterative development process, including pair-programming sessions, roles and selected meaningful version-control commits showing implementation, testing and debugging activity. Marks are not based on commit count.	10%
Packaging, reproducibility and README	The executable cards.jar, cardsTest.zip and supporting files are correctly packaged. The README gives complete, reproducible instructions to compile/run the software and execute the test suite, including relevant versions and dependencies.	5%
Individual component (20%)		
Individual code defence / live exercise	In a 5-8 minute individual conversation after the pair submission, the student demonstrates understanding of the submitted architecture/code, concurrency and thread safety, and responds to a short tracing, debugging, modification or testing task. This component is marked individually.	20%
Penalties		

Penalty	Written evidence is greater than the stated maximum of 4 pages (3-page report plus 1-page development-process record), excluding the ELE sheets.	No penalty applied, but only the first 4 pages of the written evidence will be marked.

Further Information and Guidance for students on submitting group work:

Only one student per pair should submit the completed file. Nominate someone in your pair to do this and make sure you arrange a time, before the deadline, to confirm that this has been done. If possible, do this together (or via a shared screen in Teams) so you can all check that the submission is correct, it has been uploaded to the correct link, and that the file hasn't been corrupted.